

EXP 5: Write a Program to implement on a data set of characters the three CRC polynomials – CRC 12, CRC 16 and CRC CCIP.

Aim

To write and implement a program to calculate and verify the Cyclic Redundancy Check (CRC) for a given data set of characters using the three CRC generator polynomials and to detect errors in the transmitted data.

Theory

CRC is an error-detection technique used in data communication and computer networks to detect errors that may occur during the transmission of data. CRC is based on polynomial arithmetic over modulo-2 (binary) arithmetic, where addition and subtraction are performed using the XOR operation.

In CRC, the sender and receiver agree upon a generator polynomial. The generator polynomial is fixed for a particular CRC standard and determines the error-detection capability of the CRC. For a generator polynomial of degree r , the sender appends r zeros to the data and performs modulo-2 division by the generator polynomial. The remainder obtained from this division is called the CRC checksum or CRC remainder.

The CRC remainder is appended to the original data to form the transmitted codeword.

Transmitted Codeword = Data + CRC Remainder

At the receiver, the complete received codeword is divided by the same generator polynomial. If the remainder is zero, the receiver assumes that no detectable error has occurred. If the remainder is non-zero, an error is detected.

Generator Polynomials

For this experiment, the following generator polynomials are used:

1. CRC-12

$$G(x) = x^{12} + x^{11} + x^3 + x^2 + x + 1 \Rightarrow 1100000001111$$

2. CRC-16

$$G(x) = x^{16} + x^{15} + x^2 + 1 \Rightarrow 110000000000000101$$

3. CRC-CCIP

$$G(x) = x^{16} + x^{12} + x^5 + 1 \Rightarrow 10001000000100001$$

Step-by-Step Algorithm:

1. Read the binary input data and key (generator polynomial).
2. Append $(n-1)$ zeros to the data, where $n = \text{len}(\text{key})$.
 - $\text{appended_data} = \text{data} + '0' * (n - 1)$
3. Initialize tmp with the first n bits of appended_data.
4. Repeat until all bits of appended_data are processed:
 - If the first bit of tmp is 1:
 - $\text{tmp} = \text{XOR}(\text{tmp}, \text{key}) + \text{next bit}$
 - Else:
 - $\text{tmp} = \text{XOR}(\text{tmp}, '0'*n) + \text{next bit}$
 - Discard the first bit of tmp to maintain size n.
5. After loop, perform one final XOR:
 - If the first bit of tmp is 1:
 - $\text{tmp} = \text{XOR}(\text{tmp}, \text{key})$
 - Else:
 - $\text{tmp} = \text{XOR}(\text{tmp}, '0'*n)$
6. The resulting tmp is the remainder (CRC).
7. Final transmitted codeword = data + remainder

PART 2: Receiver-Side CRC Verification Algorithm

1. Initialize with the first n bits of received_codeword.
2. Repeat modulo-2 division similar to sender-side:
 - If first bit of tmp is 1:
 - $\text{tmp} = \text{XOR}(\text{tmp}, \text{key}) + \text{next bit}$
 - Else:
 - $\text{tmp} = \text{XOR}(\text{tmp}, '0'*n) + \text{next bit}$
 - Shift left: discard first bit to keep tmp at size n
3. After loop, perform one final XOR as before.
4. Check the final remainder (tmp):
 - If it is all '0's → No Error
 - Else → Error Detected

Program

```
def xor(a, b):
    result = ''
    for i in range(1, len(b)):
        result += '0' if a[i] == b[i] else '1'
    return result

# Perform Modulo-2 Division
def mod2div(dividend, divisor):
    pick = len(divisor)
    tmp = dividend[0:pick]

    while pick < len(dividend):
        if tmp[0] == '1':
            tmp = xor(divisor, tmp) + dividend[pick]
        else:
            tmp = xor('0' * pick, tmp) + dividend[pick]
        pick += 1

    # Final XOR step
    if tmp[0] == '1':
        tmp = xor(divisor, tmp)
    else:
        tmp = xor('0' * pick, tmp)

    return tmp

# Sender-side: Encode data with CRC
def encodeData(data, key):
    l_key = len(key)

    # Append zeros to the data
    appended_data = data + '0' * (l_key - 1)

    # Calculate CRC remainder
    remainder = mod2div(appended_data, key)

    # Create transmitted codeword
    codeword = data + remainder

    return codeword, remainder

# Receiver-side: Check for error
def checkData(received_codeword, key):
    remainder = mod2div(received_codeword, key)
```

```

        if '1' in remainder:
            print("Error Detected")
        else:
            print("No Error Detected")

# Dynamic Input

data = input("Enter the data bits: ")
key = input("Enter the generator polynomial (key): ")

# Validate input
if not all(bit in '01' for bit in data):
    print("Invalid data! Enter only 0s and 1s.")

elif not all(bit in '01' for bit in key):
    print("Invalid key! Enter only 0s and 1s.")

else:
    # Sender side
    transmitted_codeword, crc = encodeData(data, key)

    print("\n--- Sender Side ---")
    print("Original Data:      ", data)
    print("Generator Key:        ", key)
    print("CRC Remainder:         ", crc)
    print("Transmitted Code:      ", transmitted_codeword)

    # Receiver side
    print("\n--- Receiver Side ---")

    received_codeword = input("Enter the received codeword: ")

    if not all(bit in '01' for bit in received_codeword):
        print("Invalid received codeword!")

    else:
        print("Received Codeword: ", received_codeword)
        checkData(received_codeword, key)

```

Output: